Helmet Detection System Using YOLOv8

(AI Internship Responsive Interface System) Task 1 | MUHAMMAD HASSAN IFTIKHAR

1. Project Overview
I built an object-detection system with YOLOv8 that detects helmets, no-helmet cases and persons in road and workplace images. Wearing a helmet is a basic safety rule on roads and construction sites, so automatic detection can support safety monitoring. The goal was to learn the full workflow: data collection, annotation, training, evaluation, inference and error analysis. The final system draws bounding boxes with class labels and confidence scores on unseen images.

2. Tools and Technologies
Python (Code on Colab)
Google Colab (For Tesla T4 GPU I used Google Colab)
Google Drive (used for storage dataset, images, results)
YOLOv8 & Ultralytics (for training,validations and predictions)
OpenCV (For reading the prediction images in the grid cell (`cv2.imread`) and inside Ultralytics)
Roboflow (used it for Annotations & exporting Yolov8 dataset)
Matplotlib (The loss curves, confusion matrix display and the prediction grid)

3. Dataset Preparation
I collected 227 images from public sources (a Kaggle dataset under Apache 2.0, Pexels and Unsplash), removed 3 unusable images, and manually annotated the remaining 224 in Roboflow. The three classes are helmet, no_helmet and person. I exported the annotations in YOLOv8 format and split the data into 157 training, 45 validation and 22 test images (70/20/10).

4. Model Training
I trained YOLOv8n from pretrained weights on Google Colab with a Tesla T4 GPU, image size 640x640, batch size 16 and 50 epochs. Training took 2.5 minutes. The best checkpoint was saved as best.pt. As a second experiment I trained YOLOv8s with the same settings (3.7 minutes).

5. Evaluation Results

Class	Precision	Recall	mAP@0.5	mAP@0.5:0.95
all	0.761	0.757	0.794	0.487
helmet	0.761	0.684	0.740	0.446
no_helmet	0.856	0.913	0.918	0.696
person	0.666	0.674	0.725	0.320

On the test set, YOLOv8n reached precision 0.761, recall 0.757, mAP@0.5 0.794 and mAP@0.5:0.95 0.487. On the validation set it reached 0.870, 0.803, 0.902 and 0.557.

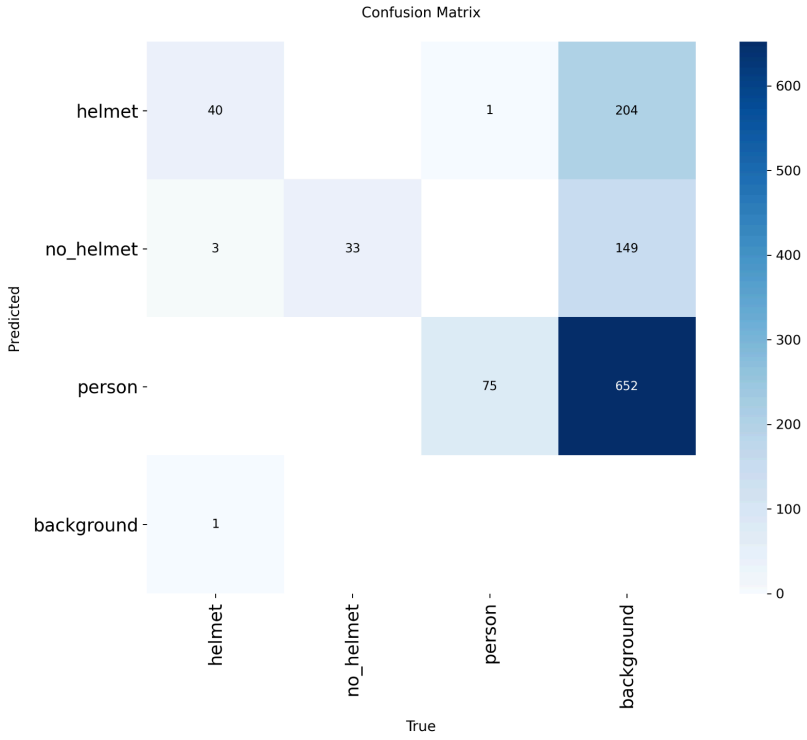


Figure 1: Confusion matrix (YOLOv8n)

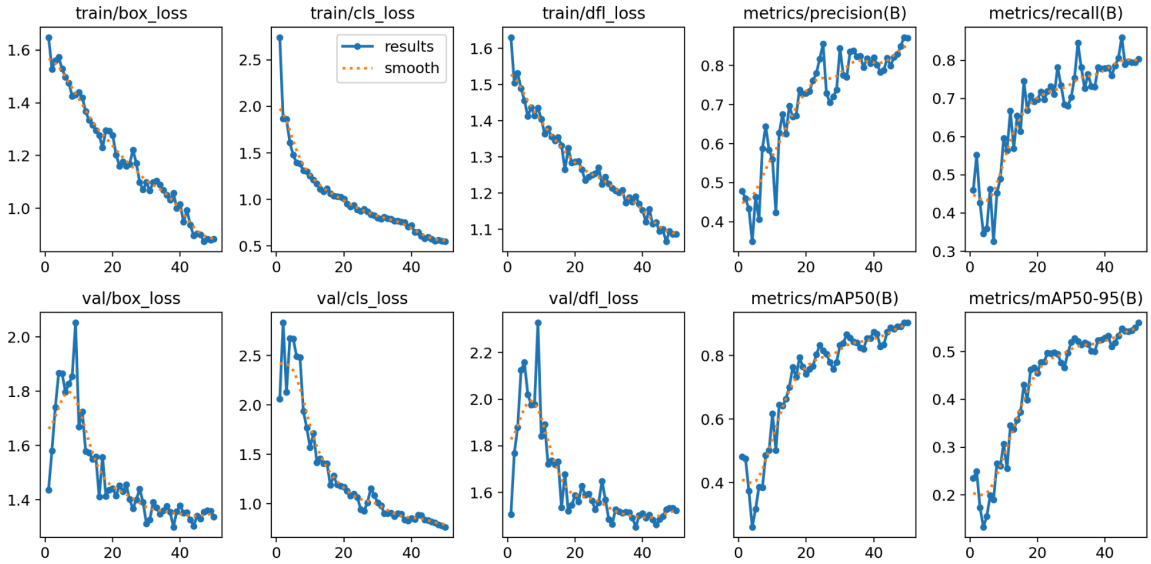


Figure 2: Training and validation loss curves

Best class: no_helmet (mAP@0.5 0.918). Weakest class: person (0.725).
- Precision: of all the objects the model predicted, how many were correct.
- Recall: of all the real objects, how many the model found.
- mAP: the average precision across classes. mAP@0.5 counts a box as correct if it overlaps the real box by at least 50%; mAP@0.5:0.95 is stricter and rewards tighter boxes.
The model made mistakes mainly on small, far-away and overlapping people, in crowded or dark scenes, and because some people were not annotated.
YOLOv8s scored similarly (test mAP@0.5 0.804), but with only 22 test images the difference is small, so I kept YOLOv8n because it is smaller and scored higher on validation.

6. Inference Results
I tested the model on the 22 unseen test images. It detected most helmets, no-helmet heads and persons, and drew boxes, labels and confidence scores.

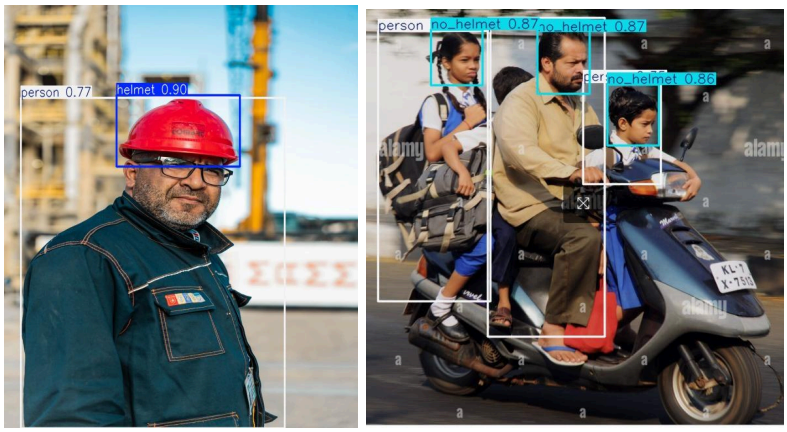


Figure 3: Predictions on unseen test images

7. Error Analysis
The test predictions had 19 missed objects, 31 extra boxes and 2 wrong-class errors. Problems: small objects, crowded scenes, poor lighting, overlapping riders and some missing annotations. The 10 examples are in error_analysis.csv.
Improvements: 1) annotate every visible person and head consistently, 2) add more crowded and low-light images, 3) raise the confidence threshold from 0.25 to 0.4, 4) train a larger model with more data.

8. What I Learned
- How to prepare a computer-vision dataset
- How to draw bounding boxes carefully in Roboflow
- The YOLO label format
- How to train YOLOv8 on Google Colab
- How to read precision, recall, mAP and the confusion matrix
- That data quality matters more than model size
- How to use error analysis to plan improvements

9. Future Improvements

Add more images and low-light examples, balance the classes, annotate more carefully, train a larger model, deploy a web demo, and test on real-time video.

10. Links

GitHub:https://github.com/Hassan-Iftikhar88/helmet-detection-yolov8.git
Drive:https://drive.google.com/drive/folders/1EaCfPv9MSTbCU8QWugFW4xZbT6YQto_g?usp=sharing
Colab: https://colab.research.google.com/drive/1ufc4JnoMXgRodw87WPs5Flytw9w8O01R